Chapter 11 Student Notes and Labs

Integrating Brain Mechanisms into AI Models: What Machines Could Borrow from Biology

Student Notes

What this chapter is for

Chapter 10 measured machine learning against the brain and sorted the differences into solutions worth borrowing and constraints not worth copying. Chapter 11 takes the borrowable solutions one at a time: what the brain does, what has been built in its image, how well it works, and what is still missing.

If you come from software or machine learning, the chapter is designed to correct three instincts:
1) treating "brain-inspired" as a claim of quality, rather than asking which principle was borrowed and whether it beat the alternative at scale,
2) treating copying the brain's mechanisms as the most faithful way to borrow, when the successes have come from borrowing principles and building them in silicon's own terms, and
3) treating learning rules as interchangeable software, when the cost of moving data makes local learning an engineering question as well as a biological one.

Your goal is to be able to explain the three ways of borrowing, why copying has mostly failed, how replay, protection, and gating fight forgetting, how retrieval and attention relate to the hippocampus and Hopfield networks, what neuromodulation could add to learning-rate schedules, why moving data dominates energy and what local rules and new hardware offer, how homeostatic reward and learning-progress curiosity resist hacking, what a cheap monitor of reliability would do, what development offers as a curriculum, and how brain data can train machines.

Copying and borrowing

- The Wrights borrowed one principle from buzzards (roll control by twisting the wings), not flapping (Kelly 1943; Wright 1900, 1901).
- Three ways to borrow (Figure 11.1): copy the mechanism; borrow the principle; borrow the data.
- Copying: Markram's 2009 claim of a simulated brain in ten years; the Human Brain Project (2013 to 2023) and Blue Brain (to 2024) left tools and data, not learning machines (Markram 2015).
- Borrowing principles: convolution from simple and complex cells; vision transformers match it once data are plentiful (Dosovitskiy 2021), so borrowed principles pay most when data are scarce.
- Convergences, not borrowings: experience replay (1992, before hippocampal replay was recorded), temporal-difference learning (from conditioning models), attention as associative memory. The brain and engineers found the same solution.

- Neuroscience validates and suggests algorithms (Hassabis 2017); it lends inductive biases (Sinz 2019).
- Test: a borrowing must beat the alternative at scale (the bitter lesson, Chapter 10).

Learning without forgetting

- Generative replay of internal representations, with no stored data (van de Ven 2020); sleep-like offline replay with a Hebbian rule (Tadros 2022).
- Protection: elastic weight consolidation, linked to spine stabilization (Kirkpatrick 2017; Yang 2009); synaptic intelligence (Zenke 2017); synapses with many timescales (Benna and Fusi 2016); metaplasticity (Laborieux 2021).
- Gating: different tasks use different dendritic branches (Cichon and Gan 2015); gate 80 percent of units per task; with protection, 95 percent across 100 tasks (Masse 2018) (Figure 11.2).
- Large models: continual fine-tuning forgets (Luo 2025); LoRA learns less and forgets less (Biderman 2024); new facts by fine-tuning raise hallucination (Gekhman 2024). Industry sidesteps forgetting: retrieval, and continued training on mixed old and new data (DeepSeek 2025).
- Choosing what to replay: reverse replay tracks reward changes (Ambrose 2016); replay by utility (Mattar and Daw 2018); prioritized replay (Schaul 2016).

A memory that learns in use

- The hippocampal index (Teyler and DiScenna 1986) and retrieval-augmented models (Lewis 2020); RETRO's two-trillion-token database matched models 25 times larger (Borgeaud 2022). Retrieval never consolidates.
- Hopfield networks (1982; Nobel Prize 2024): capacity about 0.14N (Amit 1985). Modern Hopfield networks: dense memories (Krotov and Hopfield 2016); one recall step equals attention (Ramsauer 2021) (Figure 11.3).
- Fast weights (Ba 2016; Schmidhuber 1992); linear attention as a fast-weight memory (Schlag 2021); transformers and the hippocampal formation (Whittington 2022).
- Storing by surprise: Titans (Behrouz 2025). Figure 11.4 maps the brain's memory systems onto a deployed model's.

Learning when to learn

- People learn faster when the better of two options keeps switching and more slowly when it is stable; anterior cingulate activity tracks volatility (Behrens 2007).
- Engineers fix a schedule in advance; Adam adapts step size to gradient statistics (Kingma and Ba 2015), not to change in the world. Neuromodulators as the learner's settings (Doya 2002).
- Learned plasticity (Miconi 2018) and learned neuromodulation of plasticity (Miconi 2019); meta-reinforcement learning (Wang 2018).
- A schedule fixed in advance cannot know the world has changed.

Local rules and the cost of moving data

- Horowitz (2014): a DRAM read costs hundreds of times a multiplication (Figure 11.5). Backpropagation adds stored activations and backward traffic.
- Cortex spends about 35 times more on communication than computation (Levy and Calvert 2021).
- Hardware: NorthPole (Modha 2023); analog in-memory chip (Ambrogio 2023); Hala Point (Intel 2024).
- Spikes have no useful slope; surrogate gradients (Neftci 2019) (Figure 11.6); e-prop combines eligibility traces with a broadcast learning signal (Bellec 2020), the three-factor rule of Chapter 3.
- Alternatives to backpropagation: equilibrium propagation (Scellier and Bengio 2017), forward-forward (Hinton 2022), physical networks (Wright 2022; Momeni 2025). None yet matches backpropagation at scale.
- Emergent sparsity in transformers (Li 2023).

Values that cannot be rewritten

- CoastRunners (Clark and Amodei 2016); specification gaming (Krakovna 2020).
- Reward models are proxies (Christiano 2017; Ouyang 2022); over-optimization lowers true quality (Gao 2023), Goodhart's law.
- Homeostatic reward saturates and turns negative past the set point (Keramati and Gutkin 2014) (Figure 11.7); feeling machines (Man and Damasio 2019). Borrow saturation and balance, not self-concern; the KL penalty in language-model training is a version (Ouyang 2022).
- Curiosity as prediction error (Pathak 2017) is captured by a noisy television (Burda 2019); curiosity as learning progress (Gottlieb 2013; Kidd and Hayden 2015; Schmidhuber 1991).

Knowing when to stop and think

- Confidence in rat orbitofrontal cortex (Kepecs 2008); conflict monitoring (Botvinick 2001).
- Adaptive computation (Graves 2016; Banino 2021); uncertainty from ensembles and dropout (Lakshminarayanan 2017; Gal and Ghahramani 2016).
- The missing incentive: confidence targets in evaluations (Kalai 2025).
- Typists' fingers slow after their own errors even when the screen secretly corrects them, and not after errors inserted for them (Logan and Crump 2010): a fast monitor independent of belief.

Growing up

- Blurry vision first helps face recognition across resolutions; cataract patients' configural deficits (Vogelsang 2018).
- Starting small (Elman 1993) contradicted (Rohde and Plaut 1999); curriculum learning (Bengio 2009), mixed results; large models use shifting data mixtures ending on the cleanest data (Grattafiori 2024).
- Infant video, youngest first (Sheybani 2023); 200 hours of one child's video (Orhan and Lake 2024); the embodied Turing test (Zador 2023).

Borrowing the data

- Regularizing toward mouse V1 similarity (Li 2019); a fixed V1 front end, VOneNet, resisted 51 percent of attacks against 16 percent, with a small gain against corruptions and four points lost on clean images (Dapello 2020); brain-weighted training (Fong 2018); predicting monkey V1 as a second task (Safarani 2021). Modest, small-scale, worth watching.

What building teaches biology

- Feynman: "What I cannot create, I do not understand."
- Confirmed by building: interleaving prevents forgetting; gating works, consistent with tasks separated onto dendritic branches; generated replay suffices; transformers converge on hippocampal mathematics; three-factor rules approximate gradients.
- Exposed by building: no local rule yet matches backpropagation at scale.

Practical summary

- Borrow principles, build them in silicon's terms, and test them at scale.
- Forgetting is fought by replay, protection, and gating, best in combination.
- Attention is an associative memory; what deployed models lack is consolidation and storage by surprise.
- Moving data, not arithmetic, dominates energy; local learning would let memory learn in place.
- Rewards should saturate and balance; curiosity should seek progress, not noise.
- Development offers curricula ordered by the learner; brain recordings can regularize networks.

Labs

Lab Notes: how to succeed in these labs

These labs turn the chapter's borrowings into short calculations: what it costs to move a number compared with using it, how an associative memory recalls a pattern, how gating shares a network among tasks, how a reward defined by the body's needs behaves, how a three-factor rule assigns credit late, and why curiosity should seek learning rather than surprise. Each isolates one idea. The arithmetic is light; the point is to see where the numbers come from and how much they depend on assumptions.

General rules:
1) Write every intermediate value, rounded to two decimal places unless told otherwise.
2) State every assumption you add, and say which way it would move the answer.
3) End with an interpretation: what does the calculation show, and what does it leave out?

What to hand in for every lab

- A table of values.
- A diagram or sketch where requested (a photo of a hand drawing is fine).
- A short interpretation (6-12 sentences).

Lab 11.1 - The Cost of Moving Data

Purpose

To reproduce the argument of Figure 11.5: why fetching weights, not arithmetic, dominates the energy of running a large network, and why computing where the memory is would help.

Given values

From Horowitz (2014), for a 45-nanometer chip: multiplying two 32-bit floating-point numbers costs 3.7 pJ and adding them 0.9 pJ. Reading 64 bits from off-chip memory (DRAM) costs 1.3 to 2.6 nJ; take 1 nJ for each 32-bit weight. 1 pJ = 10^-12 J; 1 nJ = 10^-9 J; 1 Wh = 3,600 J.
A network has one billion (10^9) weights. Processing one input uses each weight once: one fetch, one multiplication, and one addition.

Procedure

1) Compute the energy for one input spent on fetching weights and on arithmetic. What is the ratio? (Check: 1 J; 0.0046 J; about 217.)
2) If the network processes a batch of inputs together, each weight is fetched once and used for every input in the batch. Compute the energy per input for batches of 1, 64, and 1,000. (Check: 1.00 J, 0.020 J, 0.0056 J.)
3) How many inputs of batch size 1 could be processed with 0.3 Wh? (Check: about 1,075.)
4) Batching helps a data center, which serves many users at once. Explain why it helps a brain, or a robot acting in real time, much less.
5) Suppose a chip stores each weight beside the unit that uses it, so that no fetch is needed. What is the energy per input now, with batch size 1? What has been given up? (Consider how many weights fit on a chip, and Chapter 10's comparison of brain and silicon.)
6) Write 6-10 sentences: relate this calculation to Levy and Calvert's estimate that the cortex spends about 35 times more energy on communication than on computation. In what sense do brains and chips face the same problem?

Lab 11.2 - An Associative Memory by Hand

Purpose

To see recall by pattern completion in a classical Hopfield network, and the same recall done by attention over stored patterns, as in Figure 11.3.

Given values

Six units, each +1 or −1. Two stored patterns:
p1 = (+1, +1, +1, −1, −1, −1)
p2 = (+1, −1, +1, −1, +1, −1)
Classical weights: w_ij = p1_i p1_j + p2_i p2_j for i ≠ j, and w_ii = 0. A unit's new state is the sign of the weighted sum of the others.

Procedure

1) Write out the 6 × 6 weight matrix. (Check: the first row is 0, 0, 2, −2, 0, −2.)

2) Cue the network with p1 with its first unit reversed: (−1, +1, +1, −1, −1, −1). Compute the weighted sum for every unit and the new states. (Check: sums 6, 2, 2, −2, −2, −2; the new state is p1.)
3) Now recall by attention. Compute the overlap of the cue with each stored pattern (the sum of products of matching units). (Check: 4 and 0.)
4) Turn the overlaps into weights with a softmax, $w_k = \exp(\beta s_k) / \Sigma \exp(\beta s_j)$, for $\beta = 0.5$ and $\beta = 2$. Compute the weighted average of the stored patterns. (Check: $\beta = 0.5$ gives weights 0.88 and 0.12 and an average whose second and fifth units are ±0.76; $\beta = 2$ gives weights of essentially 1 and 0 and returns p1.)
5) Taking the sign of the weighted average, does attention recall p1 in both cases? What does β control?
6) The classical network can hold about 0.14×6, less than one random pattern reliably, yet it recalled p1 here. Why? (Look at how different p1 and p2 are.) What would happen with a third pattern close to p1?
7) Write 6-10 sentences: why a transformer's context window can be read as an associative memory written in one shot, and what it lacks compared with the hippocampus (consider consolidation into the slow weights).

Lab 11.3 - Sharing a Network Among Tasks

Purpose

To work out the arithmetic of the context gating in Figure 11.2.

Given values

A network has 200 hidden units. Each task uses a fixed random 20 percent of them (each unit included with probability 0.2, independently), and the rest are silenced for that task.

Procedure

1) How many units does a task use on average? How many does it share with one other task, on average? What fraction of its own units is that? (Check: 40; 8; 20 percent.)
2) With five tasks, what fraction of a task's units are shared with at least one of the other four? (Check: $1 - 0.8^4 \approx 0.59$.)
3) What fraction of all units has never been used after 10 tasks? After 20? (Check: about 0.11; about 0.012.) What does this predict about how forgetting under gating changes as tasks accumulate?
4) In Figure 11.2, protecting important weights did nothing on its own but helped greatly when combined with gating. Using your answers, explain why: what does gating do to the number of weights each task needs to protect?
5) A brain cannot know in advance how many tasks it will face. Suggest one way the fraction of units used per task might be set adaptively, and one cost of making it smaller.
6) Write 6-10 sentences relating gating to Cichon and Gan's finding that different motor tasks strengthen synapses on different dendritic branches of the same neurons.

Lab 11.4 - A Reward the Body Defines

Purpose

To compute the homeostatic reward of Figure 11.7 and see why it cannot be hacked by taking more.

Given values

The internal state h is the energy store relative to its set point (0 is the set point). The drive is D(h) = |h| ^1.5. A meal raises h by 2. The reward of a meal is D(h) − D(h + 2), the reduction in drive it produces.

Procedure

1) Compute the reward of one meal at h = −6, −2, −1, 0, and +2. (Check: 6.70; 2.83; 0.00; −2.83; −5.17.)
2) At which state does a meal stop being rewarding? Why is it below the set point rather than at it? (Hint: the meal is large compared with the remaining deficit.)
3) Compare a fixed reward of 4 per meal. An agent that maximizes the total of either reward is offered unlimited food. Describe what each agent does, and connect your answer to the CoastRunners boat.
4) Keramati and Gutkin proved that maximizing this reward is the same as keeping the internal state near its set point. Explain in two or three sentences why this is close to true by definition, and why it is nonetheless useful.
5) Design a reward for a cleaning robot with two internal variables, battery charge and the dirt in its bin, each with a set point, so that it neither overcharges nor overfills. Write the drive and the reward.
6) Write 6-10 sentences: the chapter argues that the property worth borrowing is saturation and balance, not self-concern. Give one danger of a machine that protects its own internal state, and one way your design in step 5 avoids it.

Lab 11.5 - Credit Assigned Late: a Three-Factor Rule

Purpose

To compute the weight change under a three-factor rule with an eligibility trace, the local learning scheme behind e-prop and the dopamine-gated plasticity of Chapter 3.

Given values

A synapse's eligibility trace jumps by 1 at each pairing of presynaptic and postsynaptic activity and decays as e^(−t/τ) with τ = 1 s. A learning signal of strength 1 arrives once; the weight changes by Δw = η × (learning signal) × (trace at that moment), with η = 0.1.

Procedure

1) Pairings occur at t = 0 and t = 0.5 s, and the learning signal arrives at t = 1.5 s. Compute the trace at 1.5 s and Δw. (Check: e^(−1.5) + e^(−1.0) ≈ 0.591; Δw ≈ 0.059.)
2) Repeat with the learning signal arriving at t = 3.5 s. (Check: Δw ≈ 0.008.) What does τ set?

3) A second synapse was paired only at t = 1.4 s. Compute its Δw for a signal at 1.5 s. Which synapse gains more, and is that what you would want? (Check: e^(−0.1) ≈ 0.905; Δw ≈ 0.090.)
4) Backpropagation would compute each synapse's exact contribution to the error. Name two things the three-factor rule does not know that backpropagation does, and one thing it needs that backpropagation does not (consider memory of the past and the cost of sending errors backward).
5) Write 6-10 sentences: in e-prop, the broadcast signal is computed for each neuron from the network's error. In the brain, dopamine is broadcast to many neurons at once. What is lost when one signal must serve many neurons, and what does the eligibility trace recover?

Lab 11.6 - Curiosity: Surprise or Progress?

Purpose

To see why rewarding unpredictability attracts an agent to noise, and why rewarding learning progress does not, as in the noisy-television problem of Burda and colleagues.

Given data

An agent can attend to one of three sources. Its prediction error on each over five visits is:

Visit	Learnable pattern	Noisy television	Already known
1	1.00	1.00	0.05
2	0.60	1.00	0.05
3	0.35	1.00	0.05
4	0.20	1.00	0.05
5	0.10	1.00	0.05

Procedure

1) Curiosity as surprise: the reward for a visit is the prediction error. Sum the reward for each source. Which does the agent prefer? (Check: 2.25; 5.00; 0.25. The television.)
2) Curiosity as learning progress: the reward is the drop in error from one visit to the next (four drops per source). Sum it for each source. Which does the agent prefer now? (Check: 0.90; 0; 0. The learnable pattern.)
3) After visit 5, which source does each kind of agent prefer? What should a learning-progress agent do once the pattern is learned?
4) Kidd and Hayden report that people are most curious about what they partly know. Which of the two rewards predicts this? Draw the curve of curiosity against prior knowledge that each predicts.
5) Write 6-10 sentences: why random noise is an extreme case of something common in the real world (consider the weather, other people, a slot machine), and why an agent that learns all its life needs curiosity that is not fooled by it.

Lab 11.7 - A Map of Borrowings (concept map)

Purpose

To assemble the chapter into one argued map.

Procedure

1) List every borrowing described in the chapter (at least twelve: for example convolution, experience replay, generative replay, sleep-like replay, weight protection, context gating, retrieval, modern Hopfield attention, fast weights, learned plasticity, surrogate gradients, e-prop, in-memory chips, homeostatic reward, curiosity, adaptive computation, curricula, neural regularization).
2) Place each in one of the three columns of Figure 11.1 (copy the mechanism, borrow the principle, borrow the data) or in its band of convergences, methods found independently and recognized in the brain afterward. Defend two placements that could go either way.
3) For each, mark its status: in use at scale, shown in research systems, or proposed. Count how many are in use at scale.
4) Draw arrows between borrowings that depend on one another (for example, choosing what to replay needs a signal of surprise; local learning rules make in-memory training possible).
5) Write 8-12 sentences: which one borrowing would most change deployed systems if it passed the bitter-lesson test, and what experiment would show that it had?